

# Day 9 — Model Selection, Metrics, Thresholds, Calibration, and Error Analysis

## Beginner-friendly summary

A model is not “good” because it has the highest accuracy, lowest RMSE, or best AUC.

A model is good when it supports the **business decision** correctly.

For example:

- If predicting monthly spend, being wrong by ₹10 lakh may matter much more than being wrong by ₹10,000.
- If detecting potentially fraudulent invoices, missing fraud may be much more expensive than investigating a false alarm.
- If finance users act differently at 20%, 50%, and 80% risk, then **well-calibrated probabilities** may matter more than raw classification accuracy.
- If the model works well overall but fails badly for one geography or department, the aggregate metric can hide a production problem.

The senior applied-scientist sequence is therefore:

```
Business decision
↓
What prediction is needed?
↓
What kinds of mistakes matter?
↓
Choose model family
↓
Choose evaluation metric
↓
Validate generalization
↓
Choose decision threshold
↓
Check probability calibration
↓
Analyze errors by segment
↓
Quantify uncertainty
↓
Explain business impact
```

The critical principle for today is:

**Model score → estimated probability/value → business decision are three different things.**

A model may rank cases extremely well but still produce poor probabilities. A model may predict probabilities well but use the wrong decision threshold. A model may have excellent

aggregate metrics but fail an important business segment.

## 1. How to choose the main alternatives

| Situation                                             | Prefer                    | Why / selection criterion                                            |
|-------------------------------------------------------|---------------------------|----------------------------------------------------------------------|
| Continuous target; simple interpretable baseline      | Linear regression         | Strong baseline when relationships are approximately additive/linear |
| Many correlated features                              | Ridge regression          | L2 regularization stabilizes coefficients                            |
| Want sparse feature selection                         | Lasso                     | L1 can drive coefficients to zero                                    |
| Monetary error where every ₹1 error matters similarly | MAE                       | Directly represents average absolute miss                            |
| Large mistakes are disproportionately costly          | RMSE                      | Squares errors and punishes large misses                             |
| Need context relative to mean predictor               | $R^2$                     | Measures variance explained; not a business loss                     |
| Binary prediction with interpretable probabilities    | Logistic regression       | Strong classification baseline                                       |
| Balanced classes and equal mistake costs              | Accuracy can be useful    | Only under relatively restrictive conditions                         |
| False positives expensive                             | Precision / specificity   | Measures alert quality / control of negatives                        |
| False negatives expensive                             | Recall                    | Measures how many true positives are found                           |
| Need balance of precision and recall                  | F1                        | Useful summary when both matter similarly                            |
| Need ranking across thresholds                        | ROC-AUC                   | Overall positive-vs-negative ranking quality                         |
| Rare positive class                                   | PR-AUC                    | Much more informative about positive-class retrieval                 |
| Business decisions depend on probability magnitude    | Calibration + Brier score | Tests whether predicted probabilities mean what they claim           |
| Mistake costs can be estimated                        | Cost-based threshold      | Usually better than blindly using 0.5                                |
| Need uncertainty for a future individual value        | Prediction interval       | Includes model uncertainty + outcome noise                           |
| Need uncertainty around expected mean                 | Confidence interval       | Estimates uncertainty in conditional mean                            |

## 2. Regression: start with the decision

Suppose finance asks:

“Predict next month's vendor spend.”

That is not yet sufficient.

We need to know what finance will **do** with the forecast.

Perhaps:

```
Forecast
  ↓
Compare with budget
  ↓
Flag large expected overspend
  ↓
Controller investigates
  ↓
Potential budget adjustment
```

Now the cost of errors matters.

### **Underprediction**

Actual:

₹15M

Forecast:

₹10M

Error:

- ₹5M

Finance may fail to prepare for the overspend.

### **Overprediction**

Actual:

₹10M

Forecast:

₹15M

Finance may unnecessarily reserve capital or escalate the department.

Those two errors may not have equal business cost.

That matters more than whether an algorithm is fashionable.

### 3. Linear regression

Linear regression models:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \epsilon$$

For example:

$$\begin{aligned} \text{Spend} = & \\ & \beta_0 \\ & + \beta_1 \text{Headcount} \\ & + \beta_2 \text{TransactionCount} \\ & + \beta_3 \text{PreviousSpend} \\ & + \epsilon \end{aligned}$$

The prediction is:

$$\hat{y} = X\hat{\beta}$$

Ordinary least squares chooses coefficients minimizing:

$$\sum_i (y_i - \hat{y}_i)^2$$

---

### 4. Linear regression assumptions

This topic is often oversimplified.

The assumptions matter differently depending on whether your goal is:

1. **prediction**, or
2. statistical inference about coefficients.

#### 4.1 Approximate linearity

We assume:

[  $E[Y|X]$  ]

can be reasonably represented by an additive linear function of the features.

A relationship such as:

Spend  $\approx$  1000  $\times$  headcount

may fit.

A highly nonlinear relationship such as:

risk suddenly increases after threshold X

may not.

Residual plots often expose this.

---

## 4.2 Independence

Observations should not contain uncontrolled dependencies.

Finance data frequently violates this.

Examples:

- repeated observations from the same vendor
- consecutive months
- cost centres inside the same department
- transactions from the same customer

If January and February observations from the same cost centre are strongly correlated, treating everything as IID can make uncertainty estimates too optimistic.

---

## 4.3 Exogeneity

Informally:

$$E[\epsilon|X] = 0$$

Features should not systematically correlate with omitted error terms.

This becomes particularly important when interpreting coefficients.

Suppose salary predicts employee spending.

A coefficient does **not** automatically mean changing salary causes spending to change.

Prediction is not causality.

---

## 5. Homoscedasticity

Classical OLS inference often assumes approximately constant residual variance:

$$\text{Var}(\epsilon|X) = \sigma^2$$

But imagine:

Small departments:  
errors around  $\pm ₹20K$

Large departments:  
errors around  $\pm ₹2M$

Residual variance clearly grows with department size.

This is **heteroscedasticity**.

It can affect:

- standard errors
- confidence intervals

- statistical tests

and it tells us something important operationally: prediction reliability changes with scale.

---

## 6. Multicollinearity

Suppose features include:

```
employee_count  
monthly_salary_cost  
annual_salary_cost
```

These variables can be strongly correlated.

Prediction might still work reasonably well, but individual coefficients can become unstable.

You may see:

```
Training sample A:  
employee coefficient = +4.2  
  
Training sample B:  
employee coefficient = -1.8
```

despite similar overall predictions.

This is one reason Ridge regression can help.

---

## 7. Normality of residuals

A common misconception is:

“Linear regression requires the target to be normally distributed.”

Not exactly.

Residual normality is mainly important for certain small-sample inferential procedures.

You can have a skewed target while a useful predictive regression model still exists.

For prediction, concentrate more on:

- residual structure
- systematic bias
- heteroscedasticity
- outliers
- generalization performance
- segment-level behavior

rather than mechanically testing whether (y) looks Gaussian.

---

## 8. Residual analysis

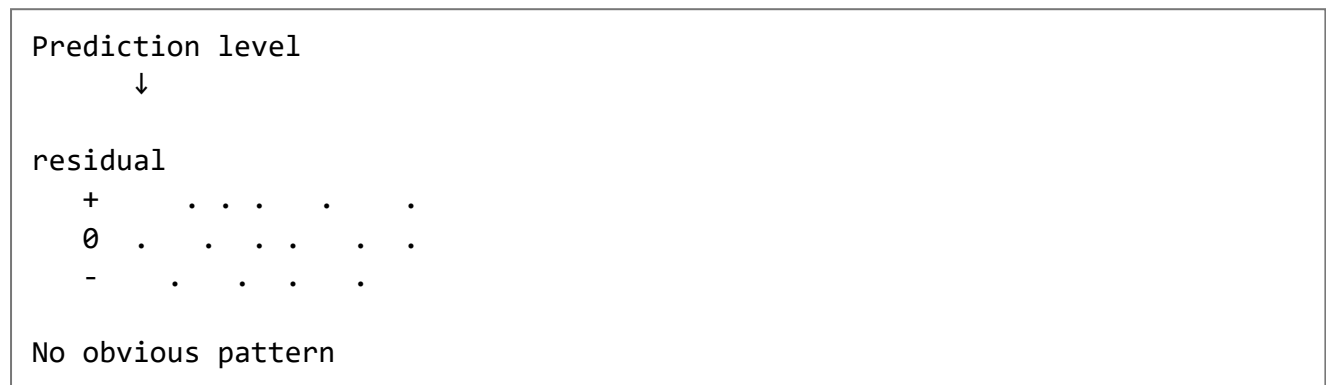
Residual:

$$e_i = y_i - \hat{y}_i$$

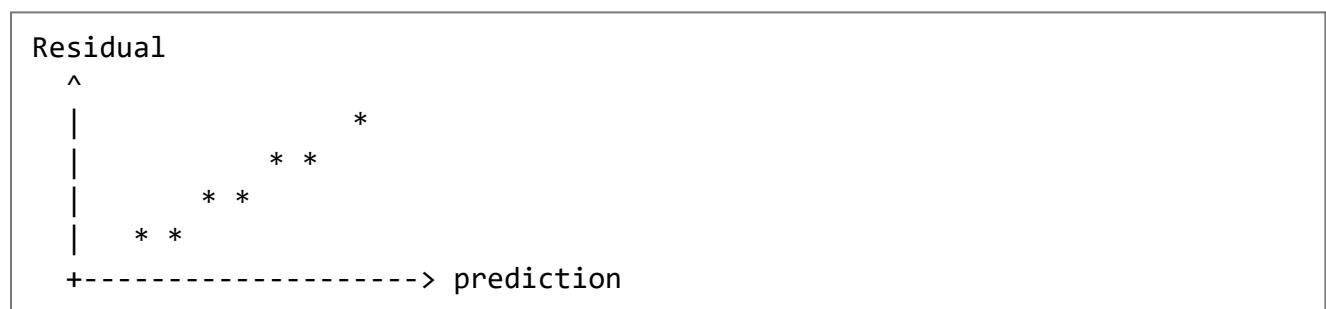
Residual analysis asks:

| What structure remains after the model has made its prediction?

Ideally:



Bad pattern:



Possible nonlinear relationship.

Another:



Possible heteroscedasticity.

---

## 9. MAE

Mean Absolute Error:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Suppose errors are:

10  
-20  
30

MAE:

$$(10 + 20 + 30)/3 = 20$$

### Interpretation

If the target is dollars:

| Average absolute prediction error is approximately \$20.

If the target is ₹ lakh:

| Average absolute error is 20 lakh.

This makes MAE very business-friendly.

### Strength

Less sensitive to extreme misses than RMSE.

### Weakness

Does not punish catastrophic errors very strongly.

## 10. RMSE

Root Mean Squared Error:

$$RMSE = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}$$

Squaring means large errors dominate.

Consider:

Errors:  
10  
10  
100

MAE:

[ 40 ]

but RMSE is much larger because of the 100 error.

Use RMSE when:

| A few huge forecast misses are particularly dangerous.

For example, treasury may care disproportionately about exceptionally large misses.

---

## 11. R-squared

$$R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$

It compares the model against predicting the target mean.

Roughly:

$R^2 = 0$   
≈ no improvement over predicting mean

$R^2$  approaching 1  
≈ explains much of observed variance

But:

**$R^2$  is not an error expressed in business units.**

A CFO probably finds:

| “Our average absolute forecast error is ₹X.”

more actionable than:

| “ $R^2$  is 0.81.”

Also remember:

$$R^2 < 0$$

is possible on unseen data when the model performs worse than the mean baseline.

---

## 12. Regularization intuition

Suppose ordinary regression learns:

feature A coefficient = 14.8  
feature B coefficient = -16.2  
feature C coefficient = 22.7

If predictors are noisy or correlated, those values can become unstable.

Regularization says:

| Fit the data, but penalize unnecessarily large coefficients.

---

## Ridge / L2

Objective:

$$RSS + \lambda \sum_j \beta_j^2$$

Large coefficients become expensive.

Ridge generally:

- shrinks coefficients
  - keeps most features
  - handles correlated predictors better
  - often improves variance/generalization
- 

## Lasso / L1

$$RSS + \lambda \sum_j |\beta_j|$$

Lasso can produce:

$$\beta_j = 0$$

for some features.

Therefore it can perform implicit feature selection.

But selection can become unstable when many features are strongly correlated.

---

## 13. Bias-variance intuition for regularization

With no regularization:

|                                       |
|---------------------------------------|
| Low bias<br>Potentially high variance |
|---------------------------------------|

With excessive regularization:

|                                             |
|---------------------------------------------|
| Higher bias<br>Low variance<br>Underfitting |
|---------------------------------------------|

The goal is not:

| maximize regularization.

It is:

| choose regularization strength that minimizes expected generalization error.

Usually through cross-validation.

---

## 14. Logistic regression

Suppose the finance problem changes.

Instead of:

| How much will this vendor overspend?

we ask:

| Is this transaction likely to require exception review?

Target:

0 = normal  
1 = exception

Now we have classification.

---

## 15. Logistic regression does not directly model class labels

First compute:

$$\begin{aligned} z = & \\ & \beta_0 + \\ & \beta_1 x_1 + \\ & \dots + \\ & \beta_p x_p \end{aligned}$$

Then:

$$p = \frac{1}{1 + e^{-z}}$$

The result is between 0 and 1.

Example:

$p(\text{exception}) = 0.78$

Then the application decides whether that probability should become an alert.

---

## 16. Odds

Probability:

[  $p$  ]

Odds:

$$\frac{p}{1-p}$$

For:

$$p = 0.8$$

odds are:

$$0.8/0.2 = 4$$

or:

|       |
|-------|
| 4 : 1 |
|-------|

Logistic regression models log-odds:

$$\log\left(\frac{p}{1-p}\right) = X\beta$$

---

## 17. Interpreting a logistic coefficient

Suppose:

$$\beta_j = 0.7$$

Then a one-unit increase in ( $x_j$ ), holding other features constant, multiplies the odds by:

$$e^{0.7}$$

approximately 2.

That means:

| odds approximately double.

It does **not** mean probability doubles.

That distinction matters.

---

## 18. Probability and threshold are separate

Suppose:

|                     |
|---------------------|
| Model output = 0.41 |
|---------------------|

Using threshold:

0.50

produces:

normal

But threshold:

0.25

produces:

review

Same model.

Same probability.

Different business decision.

Therefore:

**0.5 is not a law.**

It is merely one possible threshold.

---

## 19. Confusion matrix

For positive class = risk:

|                  | Actual |        |
|------------------|--------|--------|
|                  | Risk   | Normal |
| Predicted Risk   | TP     | FP     |
| Predicted Normal | FN     | TN     |

### TP

Correctly flagged risky case.

### FP

Normal case unnecessarily investigated.

### FN

Risky case missed.

## TN

Correctly ignored normal case.

Every classification metric is essentially emphasizing different cells of this matrix.

---

## 20. Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Seems intuitive.

But consider:

1,000,000 transactions

995,000 normal

5,000 fraud

A model predicts:

everything = normal

Accuracy:

[ 99.5% ]

Yet detected fraud:

0

This model is useless for fraud detection.

That's why accuracy can be dangerous under class imbalance.

---

## 21. Recall

$$Recall = \frac{TP}{TP + FN}$$

Question:

| Of all true risky cases, how many did we find?

High recall means few false negatives.

Use it when missing positives is dangerous.

Examples:

- fraud screening
  - safety defect detection
  - high-risk compliance cases
- 

## 22. Precision

$$Precision = \frac{TP}{TP + FP}$$

Question:

| Of everything we flagged, how much was actually positive?

Precision matters when:

- investigators have limited capacity
- manual reviews are expensive
- false alarms damage user trust

Suppose finance can review only 1,000 alerts/day.

High-recall but extremely low-precision predictions may be operationally unusable.

---

## 23. Specificity

$$Specificity = \frac{TN}{TN + FP}$$

Question:

| Of all genuinely negative cases, how many did we correctly leave alone?

Specificity is useful when false positives are expensive.

It is:

$$1 - FPR$$

where:

$$FPR = \frac{FP}{FP + TN}$$


---

## 24. F1 score

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

F1 is the harmonic mean.

It penalizes systems where one metric is strong but the other is extremely poor.

Example:

|                  |
|------------------|
| Precision = 0.95 |
| Recall = 0.10    |

F1 will remain relatively low.

### Important limitation

F1 assumes precision and recall deserve approximately symmetric treatment.

Business costs may not be symmetric at all.

Therefore a cost function can be preferable.

---

## 25. ROC-AUC

ROC curve plots:

*TPR*

against:

*FPR*

for many thresholds.

ROC-AUC can be interpreted as ranking ability:

How often does the model rank a randomly selected positive higher than a randomly selected negative?

It is threshold-independent.

Useful for comparing ranking quality.

But it can look deceptively strong under extreme class imbalance.

---

## 26. PR-AUC

Precision-recall curve plots:

|                     |
|---------------------|
| precision vs recall |
|---------------------|

across thresholds.

This focuses much more heavily on performance for the positive class.

For:

```
0.2% fraud
99.8% legitimate
```

PR-AUC is often much more informative than ROC-AUC.

Another useful fact:

The rough no-skill baseline for precision is the positive prevalence.

For example, if:

```
positive rate = 1%
```

random ranking produces roughly:

```
precision ≈ 1%
```

as its baseline.

That makes PR performance easier to interpret in imbalanced settings.

---

## 27. Never choose metrics before defining positive class

Suppose:

```
positive = fraud
```

Recall means:

```
| fraction of fraud detected.
```

But if someone reverses labels:

```
positive = legitimate
```

the same word "recall" now describes something entirely different.

Always explicitly say:

```
| The positive class is ____.
```

---

## 28. Threshold selection based on business cost

Imagine:

```
False positive:
Analyst unnecessarily reviews invoice
Cost = ₹C_FP

False negative:
```

Fraud / material error is missed  
Cost = ₹C\_FN

For threshold (t):

$$\begin{aligned} Cost(t) &= FP(t) \cdot C_{FP} \\ &+ FN(t) \cdot C_{FN} \end{aligned}$$

Choose:

$$t^* = \arg \min_t Cost(t)$$

This is much more defensible than:

threshold = 0.5

because "that's what sklearn does."

## 29. Example threshold trade-off

Conceptually:

| Threshold | Precision | Recall    | FP       | FN       | Business consequence     |
|-----------|-----------|-----------|----------|----------|--------------------------|
| 0.20      | lower     | very high | many     | few      | Large review queue       |
| 0.40      | moderate  | high      | moderate | moderate | Possible compromise      |
| 0.70      | high      | lower     | few      | many     | Risk of missed positives |

The correct row depends on the organization.

There is no universally optimal threshold.

## 30. Thresholding with operational capacity

Costs are not always easy to express in money.

Suppose:

Finance investigators can review:  
500 cases/day

Then an operational rule might be:

Select a threshold producing no more than approximately 500 alerts/day while maximizing recall.

This is a legitimate business constraint.

It turns model selection into a resource allocation problem.

## 31. Probability calibration

Suppose a model assigns:

|                                      |
|--------------------------------------|
| 100 cases probability $\approx 0.80$ |
|--------------------------------------|

If it is well calibrated, approximately:

|                 |
|-----------------|
| 80 of those 100 |
|-----------------|

should actually be positive over a sufficiently large sample.

Calibration asks:

| Does 0.8 actually mean approximately 80%?

Ranking asks something different:

| Are risky cases ranked above safe cases?

These properties are distinct.

---

## 32. Great ranking, bad calibration

A classifier could assign:

|                 |      |
|-----------------|------|
| True risky case | 0.40 |
| Normal case     | 0.20 |

and rank them correctly.

If nearly all positive cases given 0.40 actually occur only 10% of the time, probabilities are badly calibrated.

The model could still have respectable ROC-AUC.

Therefore:

|                                         |
|-----------------------------------------|
| ROC-AUC<br>≠<br>probability calibration |
|-----------------------------------------|

---

## 33. Reliability curve

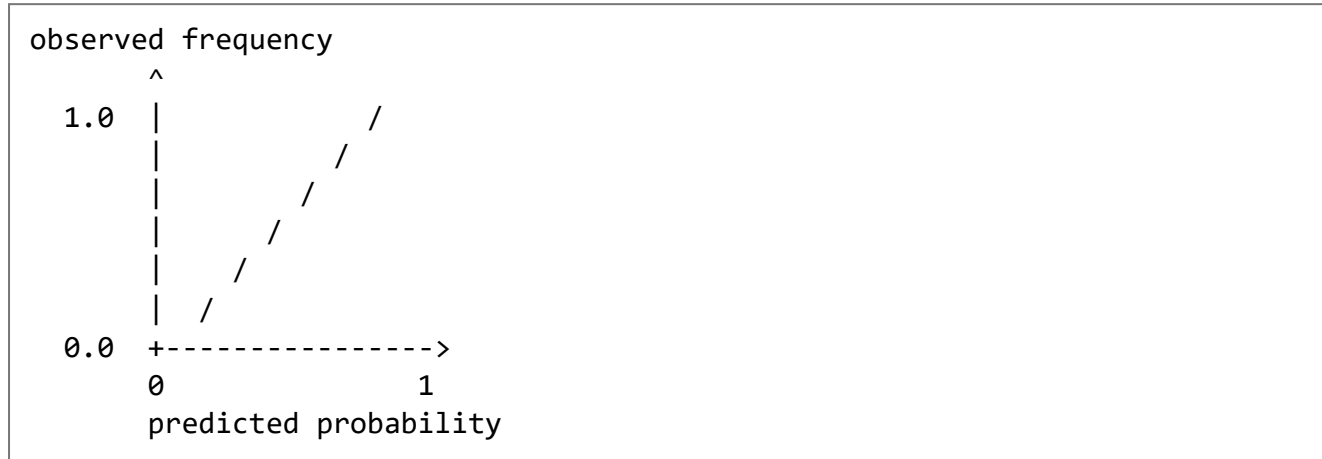
Divide predictions into probability bins.

Example:

|                       |                        |
|-----------------------|------------------------|
| Predicted probability | Observed positive rate |
| 0.10                  | $\sim 0.11$            |

|      |       |
|------|-------|
| 0.30 | ~0.28 |
| 0.50 | ~0.52 |
| 0.70 | ~0.71 |
| 0.90 | ~0.87 |

Well calibrated predictions lie roughly near:



Large systematic deviations indicate calibration problems.

### 34. Brier score

For binary classification:

$$Brier = \frac{1}{n} \sum_i (p_i - y_i)^2$$

Perfect predictions:

Brier = 0

Lower is better.

Unlike AUC, Brier evaluates the actual probability values.

Example:

Actual:

1

Prediction A:

0.90

Squared error:

$$(0.9 - 1)^2 = 0.01$$

Prediction B:

0.51

Squared error:

$$(0.51 - 1)^2 \approx 0.24$$

Both may classify as positive at threshold 0.5, but A is much better probabilistically.

---

## 35. Platt scaling

Platt scaling learns a logistic transformation of model scores.

Conceptually:

```
Raw model score
    ↓
logistic calibration model
    ↓
calibrated probability
```

It is relatively smooth and low variance.

Good when miscalibration approximately follows a sigmoid-like transformation.

In sklearn this is generally represented by:

```
method="sigmoid"
```

---

## 36. Isotonic calibration

Isotonic regression learns a flexible monotonic mapping:

```
raw score → calibrated probability
```

It assumes:

| Higher model score should not imply lower risk.

But it does not force a specific logistic shape.

### Advantage

Flexible.

### Disadvantage

Can overfit when calibration data is small.

A useful intuition:

```
Small calibration dataset
  ↓
Platt often safer

Large calibration dataset
  ↓
Isotonic becomes more viable
```

This is not an absolute rule.

---

## 37. Critical calibration leakage issue

Do not do:

```
Train classifier
  ↓
predict same training data
  ↓
fit calibrator on those predictions
```

That leaks overly optimistic model behavior into calibration.

Use:

```
Training data
  ↓
CV / held-out calibration predictions
  ↓
Calibrator
```

or calibration utilities that handle internal cross-validation appropriately.

---

## 38. Cross-validation

One train/validation split gives one estimate.

Suppose model MAE from different folds is:

```
Fold 1: 12
Fold 2: 13
Fold 3: 27
Fold 4: 11
Fold 5: 26
```

Mean:

```
17.8
```

But simply reporting:

| MAE = 17.8

hides considerable instability.

The variance is itself information.

---

## 39. Why cross-validation matters

Cross-validation helps estimate:

expected generalization  
+  
sensitivity to the sampled dataset

Standard K-fold:

Dataset  
├ Fold 1 → validation  
├ Fold 2 → validation  
├ Fold 3 → validation  
├ Fold 4 → validation  
└ Fold 5 → validation

Each row becomes validation exactly once.

---

## 40. Classification cross-validation

For classification, usually prefer:

StratifiedKFold

when IID assumptions are reasonable.

It approximately maintains class prevalence across folds.

Without stratification, rare-event datasets can accidentally produce folds with very few positives.

---

## 41. But random CV is not always correct

For financial monthly predictions:

2023  
2024  
2025  
2026

randomly mixing observations could allow future patterns into training when evaluating past observations.

In that case prefer:

```
2023 train → 2024 validate
2023-24 train → 2025 validate
2023-25 train → 2026 validate
```

That is closer to production.

Similarly, repeated observations from vendors may require:

```
GroupKFold
```

so one vendor doesn't appear in both train and validation.

Cross-validation strategy must reflect deployment reality.

---

## 42. Report variance, not only mean

Instead of:

```
| CV PR-AUC = X
```

prefer:

```
| Mean CV PR-AUC = X, with fold-to-fold standard deviation Y.
```

Or report fold values.

Large variance can indicate:

- insufficient data
  - unstable model
  - heterogeneous segments
  - distribution shifts
  - poor splitting strategy
  - rare positive events
- 

## 43. Error analysis

Aggregate score:

```
MAE = reasonable
```

doesn't guarantee reasonable performance everywhere.

Imagine:

| Department  | MAE |
|-------------|-----|
| Engineering | low |
| Finance     | low |

---

|             |                |
|-------------|----------------|
| Operations  | moderate       |
| LATAM Sales | extremely high |

The global metric may hide the LATAM problem.

## 44. Segment error analysis

Useful dimensions include:

```
department
geography
vendor
customer tier
risk band
product
time period
transaction amount
model confidence
```

For regression, inspect:

- count
- MAE
- RMSE
- mean residual/bias
- tail error
- perhaps error percentiles

For classification:

- support
- prevalence
- precision
- recall
- specificity/FPR
- FP
- FN
- cost
- predicted probability distribution

## 45. Mean residual exposes bias

Define:

$$Residual = y - \hat{y}$$

Then:

```
mean residual > 0
```

means the model systematically underpredicts.

Because actual values tend to be above predictions.

Conversely:

```
mean residual < 0
```

indicates overprediction.

MAE alone loses this direction.

Example:

```
Segment A errors:  
+100,+100,+100
```

```
Segment B:  
-100,-100,-100
```

Both:

```
MAE = 100
```

but they have completely different biases.

---

## 46. Segment metrics require sample-size awareness

Suppose:

```
Vendor A recall = 40%  
n positives = 5
```

versus:

```
Vendor B recall = 65%  
n positives = 50,000
```

Treating these as equally reliable is wrong.

Always expose:

```
support  
positive count
```

alongside segment metrics.

For tiny segments, confidence intervals or shrinkage approaches may be appropriate.

---

## 47. Error analysis by risk band

This is particularly useful.

Example:

|         |
|---------|
| 0.0–0.2 |
| 0.2–0.4 |
| 0.4–0.6 |
| 0.6–0.8 |
| 0.8–1.0 |

Within each band inspect:

|                       |
|-----------------------|
| predicted probability |
| actual event rate     |
| support               |

This combines:

- discrimination
- calibration
- operational interpretation

and can make risk scores much easier for finance teams to understand.

---

## 48. Confidence interval versus prediction interval

This distinction is essential.

Suppose we predict vendor spend.

At feature values ( $x$ ):

$$E[Y|X = x] = ₹10M$$

A **confidence interval** answers:

How uncertain are we about the mean expected spend for observations with these characteristics?

Maybe conceptually:

|                                            |
|--------------------------------------------|
| Mean expected spend:<br>₹10M               |
| 95% confidence interval:<br>₹9.7M – ₹10.3M |

---

## 49. Prediction interval

Prediction interval asks:

| Where could one new actual observation reasonably fall?

That includes:

1. uncertainty about estimated mean
2. individual outcome variability

Conceptually:

Prediction:

₹10M

95% prediction interval:

₹6M – ₹14M

Therefore typically:

$$\begin{array}{c} \textit{Prediction Interval} \\ > \\ \textit{Confidence Interval} \end{array}$$

in width.

---

## 50. Why finance usually needs prediction intervals

Suppose CFO asks:

| Could next month's spend exceed ₹13M?

A confidence interval around average expected spend may not answer that question.

They care about possible realization of the future outcome.

Prediction intervals are usually more directly relevant.

---

## 51. Modern ML complication

Classical OLS gives analytical intervals under assumptions.

For arbitrary ML models such as:

Random Forest  
XGBoost  
Neural network

prediction intervals require different methods.

Examples include:

- quantile regression
- bootstrapping
- probabilistic models
- conformal prediction

The method must match the statistical assumptions and deployment setup.

---

## 52. Explaining model performance to finance

Avoid opening with:

| ROC-AUC is 0.91 and F1 is 0.74.

A finance user thinks in decisions.

Translate technical metrics into business meaning.

For regression, useful language is:

| “For an average cost centre, the typical absolute prediction miss is approximately ₹X. Larger misses occur in segment Y, so forecasts for those units should carry a wider uncertainty range.”

For risk classification:

| “At the selected operating threshold, we catch approximately X% of historically positive cases while Y% of the alerts sent to analysts are genuine positives.”

Then:

| “Moving the threshold lower catches more cases but increases investigation workload.”

Numbers should come from the actual evaluation dataset, not be invented.

---

## 53. Explain uncertainty without making probabilities sound deterministic

Bad:

| “This vendor has an 80% fraud probability, so it is fraud.”

Better:

| “The model estimates this case at 80% risk. Among historically similar cases assigned probabilities around this range, we expect approximately that proportion to be positive if the model remains well calibrated.”

And operationally:

| “The score determines prioritization; the investigation determines the outcome.”

---

## 54. Practical task

We will build two baselines.

### Regression

Predict:

```
monthly_spend
```

Compare:

```
Dummy median baseline  
Linear regression  
Ridge regression
```

Metrics:

```
MAE  
RMSE  
R2
```

And produce error reports by:

```
department  
geography
```

### Classification

Predict:

```
requires_exception_review
```

Compare:

```
Dummy classifier  
Logistic regression
```

Metrics:

```
precision  
recall  
F1  
specificity  
ROC-AUC  
PR-AUC  
Brier score
```

Then:

```
Validation probabilities
  ↓
business cost matrix
  ↓
threshold search
  ↓
chosen threshold
  ↓
untouched test evaluation
  ↓
segment error report
```

---

## 55. Thought process before implementation

The implementation should preserve several correctness conditions.

### Decision 1 — Keep test data untouched

Do not select the threshold on test data.

Correct:

```
Train
  ↓
fit parameters

Validation
  ↓
select model
select threshold

Test
  ↓
one final evaluation
```

If you repeatedly choose thresholds based on test results, test data becomes validation data.

---

### Decision 2 — Preprocessing belongs inside the pipeline

Categorical variables need encoding.

Numeric variables may need scaling.

Wrong:

```
Fit encoder/scaler on entire dataset
  ↓
split
```

This leaks test information.

Correct:

```
Split
  ↓
pipeline.fit(train)
  ↓
pipeline.transform(validation/test)
```

---

### Decision 3 — Threshold tuning uses probabilities

Model training determines:

$$p(y = 1|x)$$

Threshold selection determines:

[ Decision(p) ]

Those should remain separate.

---

### Decision 4 — Costs belong to the business layer

We'll use placeholder values such as:

```
COST_FP = 1
COST_FN = 10
```

only to demonstrate mechanics.

They are **not project facts**.

A real project should derive these from considerations such as:

- investigation cost
  - financial exposure
  - regulatory impact
  - customer impact
  - opportunity cost
  - downstream workload
- 

## 56. Pseudocode

```
LOAD / GENERATE DATA

DEFINE:
    numeric features
    categorical features

SPLIT:
    train
```

```
validation
test
```

```
BUILD preprocessing:
    impute/scale numeric columns
    encode categorical columns
```

```
-----
REGRESSION
-----
```

```
TRAIN dummy baseline
TRAIN linear regression
TRAIN ridge regression
```

```
FOR each model:
    predict validation
    compute MAE
    compute RMSE
    compute R2
```

```
SELECT regression model
```

```
predict untouched test set
```

```
FOR each department/geography:
    count rows
    MAE
    RMSE
    mean residual
```

```
-----
CLASSIFICATION
-----
```

```
TRAIN dummy classifier
TRAIN logistic regression
```

```
FOR each model:
    get probabilities
    calculate ROC-AUC
    calculate PR-AUC
    calculate Brier score
```

```
FOR threshold from 0.01 to 0.99:
    convert probabilities to class decisions
    calculate FP
    calculate FN
```

```
cost =
    FP * false_positive_cost
```

```
+
FN * false_negative_cost
```

SELECT threshold with minimum validation cost

APPLY selected threshold to test probabilities

CALCULATE:

```
confusion matrix
precision
recall
F1
specificity
ROC-AUC
PR-AUC
Brier
```

FOR each important segment:

```
compute:
count
positive count
prevalence
precision
recall
FPR
FP
FN
business cost
```

RETURN:

```
model metrics
chosen threshold
segment reports
```

---

## 57. Python implementation

The following is deliberately self-contained and uses **synthetic data**.

```
import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

from sklearn.model_selection import train_test_split, KFold,
StratifiedKFold, cross_validate

from sklearn.dummy import DummyRegressor, DummyClassifier
```

```

from sklearn.linear_model import LinearRegression, Ridge,
LogisticRegression

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    average_precision_score,
    brier_score_loss,
    confusion_matrix,
)

RANDOM_STATE = 42
rng = np.random.default_rng(RANDOM_STATE)

N = 10_000

# -----
# 1. SYNTHETIC FINANCE-STYLE DATA
# -----

departments = np.array(
    ["Engineering", "Finance", "Sales", "Operations"]
)

geographies = np.array(
    ["India", "US", "UK", "Germany"]
)

vendor_tiers = np.array(
    ["strategic", "standard", "new"]
)

df = pd.DataFrame({
    "department": rng.choice(departments, size=N),
    "geography": rng.choice(geographies, size=N),
    "vendor_tier": rng.choice(
        vendor_tiers,
        size=N,
        p=[0.20, 0.65, 0.15],
    ),
    "headcount": rng.integers(10, 1000, size=N),
    "invoice_count": rng.integers(5, 500, size=N),
    "previous_month_spend": rng.lognormal(

```

```

        mean=11.0,
        sigma=0.65,
        size=N,
    ),
    "late_payment_rate": rng.beta(2, 15, size=N),
    "historical_variance": rng.normal(
        loc=0,
        scale=0.15,
        size=N,
    ),
})

department_effect = {
    "Engineering": 80_000,
    "Finance": 30_000,
    "Sales": 100_000,
    "Operations": 60_000,
}

geography_effect = {
    "India": 20_000,
    "US": 120_000,
    "UK": 80_000,
    "Germany": 70_000,
}

noise = rng.normal(0, 80_000, size=N)

df["monthly_spend"] = (
    25_000
    + 450 * df["headcount"]
    + 300 * df["invoice_count"]
    + 0.55 * df["previous_month_spend"]
    + df["department"].map(department_effect)
    + df["geography"].map(geography_effect)
    + noise
)

# Classification target generation.
# This creates probabilities first, then samples outcomes.

risk_score = (
    -4.0
    + 6.0 * df["late_payment_rate"]
    + 2.0 * np.abs(df["historical_variance"])
    + 0.7 * (df["vendor_tier"] == "new").astype(int)
    + 0.35 * (df["department"] == "Sales").astype(int)

```

```

)

true_probability = 1 / (1 + np.exp(-risk_score))

df["requires_exception_review"] = rng.binomial(
    1,
    true_probability,
)

# -----
# 2. FEATURES
# -----

numeric_features = [
    "headcount",
    "invoice_count",
    "previous_month_spend",
    "late_payment_rate",
    "historical_variance",
]

categorical_features = [
    "department",
    "geography",
    "vendor_tier",
]

features = numeric_features + categorical_features

# -----
# 3. PREPROCESSING
# -----

numeric_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]
)

categorical_pipeline = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(strategy="most_frequent"),
        ),
        (
            "one_hot",
            OneHotEncoder(

```

```

        handle_unknown="ignore",
        sparse_output=False,
    ),
),
]
)

preprocessor = ColumnTransformer(
    transformers=[
        ("numeric", numeric_pipeline, numeric_features),
        (
            "categorical",
            categorical_pipeline,
            categorical_features,
        ),
    ]
)

# -----
# 4. TRAIN / VALIDATION / TEST SPLIT
# -----

X = df[features]

y_reg = df["monthly_spend"]
y_clf = df["requires_exception_review"]

# Classification split is stratified.
X_train_clf, X_temp_clf, y_train_clf, y_temp_clf = (
    train_test_split(
        X,
        y_clf,
        test_size=0.40,
        stratify=y_clf,
        random_state=RANDOM_STATE,
    )
)

X_val_clf, X_test_clf, y_val_clf, y_test_clf = (
    train_test_split(
        X_temp_clf,
        y_temp_clf,
        test_size=0.50,
        stratify=y_temp_clf,
        random_state=RANDOM_STATE,
    )
)

```

```

# Regression split.
X_train_reg, X_temp_reg, y_train_reg, y_temp_reg = (
    train_test_split(
        X,
        y_reg,
        test_size=0.40,
        random_state=RANDOM_STATE,
    )
)

X_val_reg, X_test_reg, y_val_reg, y_test_reg = (
    train_test_split(
        X_temp_reg,
        y_temp_reg,
        test_size=0.50,
        random_state=RANDOM_STATE,
    )
)

# -----
# 5. REGRESSION MODELS
# -----

regression_models = {
    "dummy_median": Pipeline(
        steps=[
            ("preprocess", preprocessor),
            (
                "model",
                DummyRegressor(strategy="median"),
            ),
        ]
    ),
    "linear": Pipeline(
        steps=[
            ("preprocess", preprocessor),
            ("model", LinearRegression()),
        ]
    ),
    "ridge": Pipeline(
        steps=[
            ("preprocess", preprocessor),
            (
                "model",
                Ridge(alpha=1.0),
            ),
        ]
    )
}

```

```

    ),
}

def regression_metrics(y_true, y_pred):
    return {
        "mae": mean_absolute_error(
            y_true,
            y_pred,
        ),
        "rmse": np.sqrt(
            mean_squared_error(
                y_true,
                y_pred,
            )
        ),
        "r2": r2_score(
            y_true,
            y_pred,
        ),
    }

regression_results = []

for name, model in regression_models.items():
    model.fit(
        X_train_reg,
        y_train_reg,
    )

    val_predictions = model.predict(
        X_val_reg
    )

    metrics = regression_metrics(
        y_val_reg,
        val_predictions,
    )

    regression_results.append({
        "model": name,
        **metrics,
    })

regression_results = pd.DataFrame(
    regression_results
).sort_values("mae")

```

```

print("\nREGRESSION VALIDATION RESULTS")
print(regression_results)

# -----
# 6. SELECT REGRESSION MODEL
# -----

selected_regression_name = (
    regression_results.iloc[0]["model"]
)

selected_regression_model = (
    regression_models[selected_regression_name]
)

test_reg_predictions = (
    selected_regression_model.predict(
        X_test_reg
    )
)

print("\nREGRESSION TEST RESULTS")

print(
    regression_metrics(
        y_test_reg,
        test_reg_predictions,
    )
)

# -----
# 7. REGRESSION SEGMENT ERROR REPORT
# -----

regression_test_report = (
    X_test_reg[
        ["department", "geography"]
    ]
    .copy()
)

regression_test_report["actual"] = (
    y_test_reg.values
)

regression_test_report["prediction"] = (
    test_reg_predictions
)

```

```

regression_test_report["residual"] = (
    regression_test_report["actual"]
    -
    regression_test_report["prediction"]
)

regression_test_report["absolute_error"] = (
    np.abs(
        regression_test_report["residual"]
    )
)

regression_test_report["squared_error"] = (
    regression_test_report["residual"] ** 2
)

regression_segment_report = (
    regression_test_report
    .groupby(
        ["department", "geography"]
    )
    .agg(
        support=("actual", "size"),
        mae=("absolute_error", "mean"),
        mean_residual=("residual", "mean"),
        mse=("squared_error", "mean"),
    )
    .reset_index()
)

regression_segment_report["rmse"] = (
    np.sqrt(
        regression_segment_report["mse"]
    )
)

regression_segment_report = (
    regression_segment_report
    .drop(columns="mse")
    .sort_values(
        "mae",
        ascending=False,
    )
)

print("\nREGRESSION SEGMENT REPORT")
print(
    regression_segment_report.head(20)
)

```

```

)

# -----
# 8. CLASSIFICATION MODELS
# -----

classification_models = {

    "dummy_prior": Pipeline(
        steps=[
            ("preprocess", preprocessor),
            (
                "model",
                DummyClassifier(
                    strategy="prior",
                ),
            ),
        ],
    ),

    "logistic": Pipeline(
        steps=[
            ("preprocess", preprocessor),
            (
                "model",
                LogisticRegression(
                    max_iter=2000,
                ),
            ),
        ],
    ),
}

def probability_metrics(
    y_true,
    probabilities,
):
    return {
        "roc_auc": roc_auc_score(
            y_true,
            probabilities,
        ),

        "pr_auc": average_precision_score(
            y_true,
            probabilities,
        ),

        "brier": brier_score_loss(

```

```

        y_true,
        probabilities,
    ),
}

classification_results = []

for name, model in classification_models.items():

    model.fit(
        X_train_clf,
        y_train_clf,
    )

    probabilities = (
        model.predict_proba(
            X_val_clf
        )[:, 1]
    )

    metrics = probability_metrics(
        y_val_clf,
        probabilities,
    )

    classification_results.append({
        "model": name,
        **metrics,
    })

classification_results = pd.DataFrame(
    classification_results
).sort_values(
    "pr_auc",
    ascending=False,
)

print("\nCLASSIFICATION VALIDATION RESULTS")
print(classification_results)

# -----
# 9. SELECT CLASSIFICATION MODEL
# -----

selected_classifier_name = (
    classification_results.iloc[0]["model"]

```

```

)

classifier = (
    classification_models[
        selected_classifier_name
    ]
)

validation_probabilities = (
    classifier.predict_proba(
        X_val_clf
    )[:, 1]
)

# -----
# 10. COST-BASED THRESHOLD SEARCH
# -----

# Illustrative placeholders only.
# Replace with real business-derived costs.

COST_FALSE_POSITIVE = 1.0
COST_FALSE_NEGATIVE = 10.0

def evaluate_threshold(
    y_true,
    probabilities,
    threshold,
    cost_fp,
    cost_fn,
):
    predictions = (
        probabilities >= threshold
    ).astype(int)

    tn, fp, fn, tp = confusion_matrix(
        y_true,
        predictions,
        labels=[0, 1],
    ).ravel()

    total_cost = (
        fp * cost_fp
        +
        fn * cost_fn
    )

    precision = precision_score(

```

```

        y_true,
        predictions,
        zero_division=0,
    )

    recall = recall_score(
        y_true,
        predictions,
        zero_division=0,
    )

    specificity = (
        tn / (tn + fp)
        if (tn + fp) > 0
        else np.nan
    )

    return {
        "threshold": threshold,
        "tn": tn,
        "fp": fp,
        "fn": fn,
        "tp": tp,
        "precision": precision,
        "recall": recall,
        "specificity": specificity,
        "cost": total_cost,
    }

threshold_results = []

for threshold in np.linspace(
    0.01,
    0.99,
    99,
):
    threshold_results.append(
        evaluate_threshold(
            y_val_clf,
            validation_probabilities,
            threshold,
            COST_FALSE_POSITIVE,
            COST_FALSE_NEGATIVE,
        )
    )

threshold_results = pd.DataFrame(
    threshold_results

```

```

)

best_threshold_row = (
    threshold_results
    .sort_values("cost")
    .iloc[0]
)

best_threshold = float(
    best_threshold_row["threshold"]
)

print("\nBEST VALIDATION THRESHOLD")
print(best_threshold_row)

# -----
# 11. FINAL CLASSIFICATION TEST EVALUATION
# -----

test_probabilities = (
    classifier.predict_proba(
        X_test_clf
    )[:, 1]
)

test_predictions = (
    test_probabilities
    >= best_threshold
).astype(int)

tn, fp, fn, tp = confusion_matrix(
    y_test_clf,
    test_predictions,
    labels=[0, 1],
).ravel()

test_precision = precision_score(
    y_test_clf,
    test_predictions,
    zero_division=0,
)

test_recall = recall_score(
    y_test_clf,
    test_predictions,

```

```

        zero_division=0,
    )

test_f1 = f1_score(
    y_test_clf,
    test_predictions,
    zero_division=0,
)

test_specificity = (
    tn / (tn + fp)
)

test_roc_auc = roc_auc_score(
    y_test_clf,
    test_probabilities,
)

test_pr_auc = average_precision_score(
    y_test_clf,
    test_probabilities,
)

test_brier = brier_score_loss(
    y_test_clf,
    test_probabilities,
)

test_cost = (
    fp * COST_FALSE_POSITIVE
    +
    fn * COST_FALSE_NEGATIVE
)

print("\nFINAL CLASSIFICATION TEST METRICS")

print({
    "threshold": best_threshold,
    "precision": test_precision,
    "recall": test_recall,
    "f1": test_f1,
    "specificity": test_specificity,
    "roc_auc": test_roc_auc,
    "pr_auc": test_pr_auc,
    "brier": test_brier,
    "false_positives": fp,
    "false_negatives": fn,
    "cost": test_cost,
})

```

```

# -----
# 12. CLASSIFICATION SEGMENT REPORT
# -----

classification_test_report = (
    X_test_clf[
        ["department", "geography"]
    ]
    .copy()
)

classification_test_report["actual"] = (
    y_test_clf.values
)

classification_test_report["probability"] = (
    test_probabilities
)

classification_test_report["prediction"] = (
    test_predictions
)

def classification_segment_metrics(group):

    y_true = group["actual"].to_numpy()
    y_pred = group["prediction"].to_numpy()

    tn, fp, fn, tp = confusion_matrix(
        y_true,
        y_pred,
        labels=[0, 1],
    ).ravel()

    precision = (
        tp / (tp + fp)
        if (tp + fp) > 0
        else np.nan
    )

    recall = (
        tp / (tp + fn)
        if (tp + fn) > 0
        else np.nan
    )

    specificity = (
        tn / (tn + fp)
        if (tn + fp) > 0

```

```

        else np.nan
    )

    fpr = (
        fp / (fp + tn)
        if (fp + tn) > 0
        else np.nan
    )

    segment_cost = (
        fp * COST_FALSE_POSITIVE
        +
        fn * COST_FALSE_NEGATIVE
    )

    return pd.Series({
        "support": len(group),
        "positive_count": y_true.sum(),
        "prevalence": y_true.mean(),
        "mean_predicted_probability": (
            group["probability"].mean()
        ),
        "precision": precision,
        "recall": recall,
        "specificity": specificity,
        "fpr": fpr,
        "false_positives": fp,
        "false_negatives": fn,
        "cost": segment_cost,
    })

classification_segment_report = (
    classification_test_report
    .groupby(
        ["department", "geography"]
    )
    .apply(
        classification_segment_metrics,
        include_groups=False,
    )
    .reset_index()
    .sort_values(
        "cost",
        ascending=False,
    )
)

print("\nCLASSIFICATION SEGMENT REPORT")
print(

```

```
classification_segment_report.head(20)
)
```

---

## 58. What is non-obvious in this implementation?

### A. Why a Dummy model?

Because:

```
ML model
vs
simple baseline
```

is the meaningful comparison.

For regression:

```
DummyRegressor(strategy="median")
```

asks:

| Does the learned model actually beat the trivial strategy of predicting a constant?

For classification:

```
DummyClassifier(strategy="prior")
```

establishes what essentially prevalence-based prediction looks like.

Without the baseline, a metric can appear impressive simply because the dataset is easy or imbalanced.

---

## 59. Why Ridge after linear regression?

Linear regression gives the interpretable unregularized baseline.

Ridge asks:

| Can we obtain similar or better generalization while reducing coefficient variance?

If:

```
Linear validation MAE  $\approx$  Ridge validation MAE
```

and Ridge is more stable across folds, Ridge may be preferable.

But complexity should earn its place.

---

## 60. Why we select classification model using probabilities first

ROC-AUC, PR-AUC, and Brier don't require committing immediately to a threshold.

That lets us first answer:

```
Does the model rank well?  
Does it estimate probabilities reasonably?
```

Then separately:

```
How should the business act?
```

That separation is architecturally important.

---

## 61. Why threshold tuning occurs on validation

Suppose we tried 99 thresholds on the test set.

Although we didn't technically retrain logistic regression, we still optimized a model decision parameter against test outcomes.

Therefore we would overfit the test set.

Threshold is part of the deployed system.

Treat it like a tunable parameter.

---

## 62. Why report probability metrics after threshold selection

Changing:

```
threshold 0.3 → 0.7
```

changes:

```
precision  
recall  
F1  
specificity  
FP  
FN  
cost
```

But it does not change:

```
ROC-AUC  
PR-AUC  
Brier
```

because the underlying probability predictions are unchanged.

This distinction is extremely useful when debugging model systems.

---

## 63. Adding cross-validation

For regression:

```
ridge_cv_pipeline = Pipeline(
    steps=[
        ("preprocess", preprocessor),
        (
            "model",
            Ridge(alpha=1.0),
        ),
    ]
)

regression_cv = KFold(
    n_splits=5,
    shuffle=True,
    random_state=RANDOM_STATE,
)

scores = cross_validate(
    ridge_cv_pipeline,
    X_train_reg,
    y_train_reg,
    cv=regression_cv,
    scoring={
        "mae": "neg_mean_absolute_error",
        "r2": "r2",
    },
)

mae_per_fold = -scores["test_mae"]

print("MAE by fold:", mae_per_fold)
print("Mean MAE:", mae_per_fold.mean())
print("MAE std:", mae_per_fold.std())
```

For classification:

```
logistic_cv_pipeline = Pipeline(
    steps=[
        ("preprocess", preprocessor),
        (
            "model",
            LogisticRegression(
                max_iter=2000,
            ),
        ),
    ]
)
```

```

        ),
    ),
]
)

classification_cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=RANDOM_STATE,
)

scores = cross_validate(
    logistic_cv_pipeline,
    X_train_clf,
    y_train_clf,
    cv=classification_cv,
    scoring={
        "roc_auc": "roc_auc",
        "pr_auc": "average_precision",
        "neg_brier": "neg_brier_score",
    },
)

print(
    "PR-AUC mean:",
    scores["test_pr_auc"].mean(),
)

print(
    "PR-AUC std:",
    scores["test_pr_auc"].std(),
)

```

## 64. Calibration curves

A simple calibration diagnostic:

```

from sklearn.calibration import calibration_curve

fraction_of_positives, mean_predicted_value = (
    calibration_curve(
        y_test_clf,
        test_probabilities,
        n_bins=10,
        strategy="quantile",
    )
)

calibration_report = pd.DataFrame({
    "mean_predicted_probability":

```

```
        mean_predicted_value,  
        "observed_positive_rate":  
            fraction_of_positives,  
    })  
  
    print(calibration_report)
```

Conceptually compare:

```
Predicted 0.20  
Observed  0.19  
  
good
```

versus:

```
Predicted 0.80  
Observed  0.35  
  
seriously overconfident
```

---

## 65. Platt scaling in sklearn

Conceptually:

```
from sklearn.calibration import CalibratedClassifierCV  
  
base_classifier = Pipeline(  
    steps=[  
        ("preprocess", preprocessor),  
        (  
            "model",  
            LogisticRegression(  
                max_iter=2000,  
            ),  
        ),  
    ],  
)  
  
platt_model = CalibratedClassifierCV(  
    estimator=base_classifier,  
    method="sigmoid",  
    cv=5,  
)  
  
platt_model.fit(  
    X_train_clf,
```

```
y_train_clf,  
)
```

For isotonic:

```
isotonic_model = CalibratedClassifierCV(  
    estimator=base_classifier,  
    method="isotonic",  
    cv=5,  
)
```

Then compare:

```
Brier score  
calibration curve  
PR-AUC  
ROC-AUC
```

Calibration should improve probability quality without materially damaging ranking.

---

## 66. One subtle point: Logistic regression can already be reasonably calibrated

Logistic regression directly optimizes a probabilistic loss.

Therefore it can already produce reasonable probabilities when its functional assumptions fit the data.

You should not automatically calibrate every model.

Calibration itself can overfit.

The workflow should be:

```
Measure calibration  
    ↓  
Is there meaningful miscalibration?  
    ↓  
No → leave model alone  
  
Yes  
    ↓  
try appropriate calibration  
    ↓  
validate on unseen data
```

---

## 67. Calibration can drift

Suppose the model was trained when:

5% of invoices required review

A new control process reduces prevalence to:

2%

Even if ranking remains useful, probabilities can become miscalibrated.

Therefore production monitoring should distinguish:

ranking drift  
probability calibration drift  
threshold/business-cost drift

They are separate problems.

---

## 68. Prediction intervals: simple OLS example

For classical linear regression, statsmodels can distinguish mean-confidence intervals from observation-prediction intervals.

```
import statsmodels.api as sm

X_simple = df[
    [
        "headcount",
        "invoice_count",
        "previous_month_spend",
    ]
]

X_simple = sm.add_constant(
    X_simple
)

y_simple = df[
    "monthly_spend"
]

ols_model = sm.OLS(
    y_simple,
    X_simple,
).fit()

new_case = pd.DataFrame({
    "const": [1],
    "headcount": [300],
    "invoice_count": [150],
    "previous_month_spend": [500_000],
```

```
})

prediction = (
    ols_model
    .get_prediction(new_case)
    .summary_frame(alpha=0.05)
)

print(prediction)
```

You will see columns conceptually similar to:

```
mean
mean_ci_lower
mean_ci_upper
obs_ci_lower
obs_ci_upper
```

Where:

```
mean_ci
=
confidence interval for expected mean

obs_ci
=
prediction interval for a new observation
```

Usually:

```
obs_ci much wider
```

---

## 69. Production trade-offs

### Logistic regression versus more complex classifier

Logistic regression offers:

- fast training
- fast inference
- explainability
- probabilistic output
- strong baseline
- easy deployment

But it may underfit:

- nonlinear interactions
- threshold effects

- complex feature interactions

A boosted-tree model may improve predictive performance.

But then ask:

How much improvement?  
At what operational complexity?  
Does calibration worsen?  
Does inference cost increase?  
Can we explain failures?  
Does segment performance improve?

Not:

XGBoost is more advanced, therefore use XGBoost.

---

## 70. Global metric versus segment metrics

Model A:

Global PR-AUC: higher

Model B:

Global PR-AUC: slightly lower  
but consistently strong across critical regions

If Model A fails badly in a strategically important geography, Model B may be safer.

That is model selection based on the business deployment context.

---

## 71. Precision-recall trade-off is operational

Imagine lowering threshold gives:

Recall:  
70% → 94%

Great.

But alerts increase:

1,000/day → 25,000/day

with investigators capable of processing:

2,000/day

The higher-recall system is not automatically better.

Those unreviewed alerts may effectively become missed cases anyway.

---

## 72. Failure mode: optimizing the wrong metric

Business problem:

| Avoid large cash-flow surprises.

Team optimizes:

$R^2$

A model improves:

$R^2 \ 0.80 \rightarrow 0.83$

but catastrophic large-value errors increase.

The metric and decision are misaligned.

Potentially choose:

RMSE  
tail error  
weighted error  
high-value-segment MAE

instead.

---

## 73. Failure mode: hidden prevalence changes

Suppose training data:

positive rate = 10%

Production:

positive rate = 1%

Precision can fall dramatically even when sensitivity and specificity remain similar.

This is why metrics must be interpreted with the operating distribution.

---

## 74. Failure mode: threshold hard-coded forever

Imagine:

```
if risk_probability >= 0.5:  
    investigate()
```

for three years.

During that time:

- fraud costs change
- investigator capacity changes
- prevalence changes
- controls improve
- regulations change

The optimum operating threshold may change.

The threshold should be treated as a versioned business policy/configuration, not an eternal property of the model.

---

## 75. Failure mode: threshold and model version disconnected

Production should not merely record:

```
prediction = risk
```

It should ideally preserve:

```
model_version  
probability  
threshold_version  
threshold  
decision
```

Why?

Because:

```
probability = 0.43
```

may produce:

```
normal under threshold 0.50
```

but:

```
review under threshold 0.30
```

Auditing requires both model and decision policy.

---

## 76. Failure mode: segment averages without support

Bad report:

Germany precision = 100%

Maybe Germany had:

1 alert

That is not robust evidence.

Always accompany segment metrics with:

n  
positive count  
predicted-positive count

---

## 77. Failure mode: treating probabilities as certainty

Model output:

0.91

doesn't mean:

| We are 91% certain this invoice is fraudulent.

It means, approximately:

| Under the model and data-generating assumptions, this observation received an estimated probability of 0.91.

Whether that number deserves frequentist interpretation depends heavily on calibration and distribution stability.

---

## 78. Failure mode: ignoring asymmetric regression costs

MAE and RMSE treat:

+₹1M

and:

-₹1M

symmetrically.

But finance may say:

| Underpredicting spend is three times more damaging than overpredicting spend.

Then standard MAE is misaligned.

You may need an asymmetric business loss.

Conceptually:

$$Loss = \begin{cases} 3|e| & \text{if underprediction} \\ |e| & \text{if overprediction} \end{cases}$$

Business loss need not be a textbook ML metric.

---

## 79. Senior-level model-selection framework

Before training anything, write these six things down.

### 1. Prediction

What exactly are we estimating?

```
monthly expense amount  
exception probability  
default probability
```

### 2. Decision

What happens because of the prediction?

```
approve automatically  
route to analyst  
increase reserve  
request documentation
```

### 3. Error types

What can go wrong?

```
underforecast  
overforecast  
  
false positive  
false negative
```

### 4. Cost

What does each mistake cost?

Not necessarily only dollars.

Could include:

- analyst hours
- customer friction
- regulatory risk
- missed revenue
- capital requirements

## 5. Metric

Choose metrics representing those costs.

## 6. Threshold

Choose the operating point representing the business trade-off.

That's the applied-scientist sequence.

---

## 80. How I would present the classification result to finance

Do not start with six ML metrics.

Start with:

“The model produces a risk probability for each transaction. We evaluated several operating thresholds against the cost of missed exceptions and unnecessary reviews. The selected threshold minimizes that validation cost under the current assumptions.”

Then:

“At this operating point, we report the fraction of true exceptions detected, how many analyst alerts are genuine, the resulting review volume, and the estimated cost of mistakes.”

Only after that add:

“ROC-AUC and PR-AUC measure ranking quality, while Brier score and calibration curves tell us whether the probabilities themselves are reliable.”

That's much closer to how technical performance connects to finance decisions.

---

## 81. How I would present regression performance

Instead of:

“ $R^2 = X$ .”

Use:

“The model's typical absolute forecast miss is ₹X. Because large misses matter materially, we also monitor RMSE and the upper tail of the error distribution. Segment analysis shows whether specific departments or regions have materially larger errors.”

Then uncertainty:

“For individual forecasts we provide prediction intervals rather than interpreting the point forecast as guaranteed spend.”

And model limitation:

“These intervals assume future data remains sufficiently similar to the population represented in evaluation.”

---

## 82. Day 9 mental model

When you see:

REGRESSION

think:

What is the cost of forecast error?  
↓  
MAE vs RMSE vs custom loss  
↓  
Residual analysis  
↓  
Segment bias  
↓  
Uncertainty interval

When you see:

CLASSIFICATION

think:

What is positive?  
↓  
How imbalanced?  
↓  
Ranking metrics  
ROC-AUC / PR-AUC  
↓  
Probability quality  
Brier / calibration  
↓  
Business decision  
threshold  
↓  
FP / FN costs  
↓  
Segment analysis

And when someone says:

“The model has 95% accuracy.”

your immediate questions should be:

What is the class prevalence?  
  
Which class is positive?

What are precision and recall?

What are FP and FN costs?

What threshold produced this accuracy?

How is PR-AUC?

Are probabilities calibrated?

How stable is performance across CV folds?

How does it perform by important segment?

That is the difference between **evaluating an ML model** and **evaluating an ML decision system**.

## Day 9 DSA — Binary Search

### 1. Beginner-friendly summary

Binary search is useful when the search space is **ordered** or when a condition changes monotonically from one state to another.

Instead of checking every element:

1 → 2 → 3 → 4 → ... → n

binary search repeatedly eliminates approximately half:

Entire search space  
↓  
half  
↓  
quarter  
↓  
eighth  
↓  
...

This gives:

$$O(\log n)$$

instead of:

[  $O(n)$  ]

The four patterns to learn are:

1. **Exact match** — find a target.

2. **Boundary search** — find first/last occurrence.
  3. **Lower/upper bound** — find an insertion boundary.
  4. **Binary search on a monotonic condition** — search an answer space instead of an array.
- 

## 2. Recognition signals

Think **binary search** when you see these signals.

### Signal 1 — Sorted input

[2, 5, 8, 11, 17, 24, 39]

and the question asks:

Does 17 exist?  
Where is 17?

Strong binary-search signal.

---

### Signal 2 — First or last occurrence

Example:

[1, 2, 2, 2, 2, 5, 9]

Question:

Find first 2.  
Find last 2.

You need **boundary binary search**.

---

### Signal 3 — Insertion position

[1, 4, 7, 10]

Question:

| Where should 6 be inserted while keeping the array sorted?

This is a **lower-bound** style problem.

---

### Signal 4 — "Minimum X such that..."

Examples:

| Find the minimum speed that finishes the work within 8 hours.

- | Find the smallest capacity capable of shipping everything within D days.
- | Find the minimum value satisfying some constraint.

This frequently means:

```
False False False True True True
                ^
            first True
```

Binary search can locate the transition.

### Signal 5 — "Maximum X such that..."

Similarly:

```
True True True True False False
                ^
            last True
```

Binary search can find the boundary.

## 3. The fundamental requirement

Binary search does **not** fundamentally require an array.

It requires a search space in which we can decide:

- | Which half can safely be discarded?

For ordinary array binary search, sorting gives us that property.

For binary search on an answer, a **monotonic condition** gives us that property.

## 4. Exact-match binary search

Suppose:

```
nums = [3, 7, 11, 15, 20, 27, 31]
target = 20
```

Start:

```
low                high
↓                  ↓
[3, 7, 11, 15, 20, 27, 31]
                ↑
                mid
```

nums[mid] = 15.

Because:

```
15 < 20
```

everything at or left of 15 can be discarded.

```

        low        high
        ↓          ↓
[3, 7, 11, 15, 20, 27, 31]
                ↑
                mid
```

Continue until:

```
nums[mid] == target
```

or the search space becomes empty.

genui☆{"computing\_algorithms\_search\_traversal\_learning\_block\_staging":  
{"type\_id":"BINARY\_SEARCH"}}↕

---

## 5. Standard exact-match template

```
def binary_search(nums, target):
    left = 0
    right = len(nums) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if nums[mid] == target:
            return mid

        if nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

The key invariant is:

| If the target still exists, it must remain somewhere inside [left, right].

---

## 6. Why left <= right?

We are using a **closed interval**:

```
[left, right]
```

Both endpoints are candidates.

If:

```
left == right
```

there is still exactly one candidate.

Therefore:

```
while left <= right:
```

is correct.

Only after:

```
left > right
```

is the search space empty.

---

## 7. Why use this midpoint calculation?

```
mid = left + (right - left) // 2
```

In Python:

```
(left + right) // 2
```

is also safe because Python integers do not overflow.

But in fixed-width languages, `left + right` could theoretically overflow.

The first form is therefore a useful language-independent habit.

---

## 8. Exact-match brute-force reasoning

Without binary search:

```
def search(nums, target):
    for i, value in enumerate(nums):
        if value == target:
            return i

    return -1
```

## Complexity

Time:

[  $O(n)$  ]

Space:

[  $O(1)$  ]

If there are one million items, we might inspect nearly one million values.

Binary search requires only roughly:

$$\log_2(1,000,000) \approx 20$$

iterations.

That is the power of halving.

---

## 9. The most important binary-search distinction

There are two fundamentally different goals.

### Goal A

```
Find target
```

You can immediately return when:

```
nums[mid] == target
```

### Goal B

```
Find first/last position satisfying something
```

You **cannot necessarily return immediately** after finding a valid candidate.

You must continue searching for a better boundary.

This distinction causes many binary-search bugs.

---

## 10. Boundary search

Consider:

```
[2, 4, 4, 4, 4, 7, 9]
```

Suppose:

```
target = 4
```

Any of these indices:

```
1, 2, 3, 4
```

is technically an exact match.

But if the question asks:

| Find the first 4.

finding index 3 is not enough.

You must continue left.

---

## 11. Finding the first occurrence

Conceptually:

```
if nums[mid] == target:
    remember mid
    search LEFT
```

because there might be another target earlier.

```
def first_occurrence(nums, target):
    left = 0
    right = len(nums) - 1
    answer = -1

    while left <= right:
        mid = left + (right - left) // 2

        if nums[mid] == target:
            answer = mid
            right = mid - 1

        elif nums[mid] < target:
            left = mid + 1

        else:
            right = mid - 1

    return answer
```

Notice:

```
answer = mid
right = mid - 1
```

We save the candidate but keep investigating left.

---

## 12. Finding the last occurrence

Reverse the boundary direction.

```
def last_occurrence(nums, target):
    left = 0
    right = len(nums) - 1
    answer = -1

    while left <= right:
        mid = left + (right - left) // 2

        if nums[mid] == target:
            answer = mid
            left = mid + 1

        elif nums[mid] < target:
            left = mid + 1

        else:
            right = mid - 1

    return answer
```

When we find the target:

```
left = mid + 1
```

because we're looking for a later occurrence.

---

## 13. Lower bound

The **lower bound** of target is:

| The first index whose value is **greater than or equal to target**.

Mathematically:

```
first index i where nums[i] >= target
```

Example:

```
nums = [1, 3, 3, 3, 7, 10]
```

For:

```
target = 3
```

lower bound:

```
index = 1
```

because:

```
nums[1] = 3
```

is the first value  $\geq 3$ .

---

## 14. Lower bound when target doesn't exist

Example:

```
nums = [1, 3, 7, 10]  
target = 5
```

First element satisfying:

```
value  $\geq$  5
```

is:

```
7
```

at index:

```
2
```

So lower bound also tells us:

| Where could 5 be inserted?

```
[1, 3, 5, 7, 10]  
      ^
```

---

## 15. Lower-bound template

A particularly clean template uses a half-open interval:

```
[left, right)
```

```
def lower_bound(nums, target):  
    left = 0  
    right = len(nums)  
  
    while left < right:  
        mid = left + (right - left) // 2
```

```
        if nums[mid] < target:
            left = mid + 1
        else:
            right = mid

    return left
```

At completion:

```
left == right
```

That index is the first location satisfying:

```
nums[i] >= target
```

---

## 16. Why `right = mid`, not `mid - 1`?

Suppose:

```
nums[mid] >= target
```

`mid` itself may be the answer.

Therefore we cannot discard it.

So:

```
right = mid
```

retains `mid` inside the candidate region.

This is one reason mixing binary-search templates causes bugs.

In the standard closed-interval template:

```
[left, right]
```

we often write:

```
right = mid - 1
```

In half-open boundary search:

```
[left, right)
```

we often write:

```
right = mid
```

Pick one template and understand its invariant.

---

## 17. Upper bound

Upper bound means:

| First element strictly greater than target.

```
first i such that nums[i] > target
```

Example:

```
nums = [1, 3, 3, 3, 7, 10]
```

For:

```
target = 3
```

upper bound:

```
index = 4
```

because:

```
nums[4] = 7
```

is the first value greater than 3.

---

## 18. Upper-bound implementation

```
def upper_bound(nums, target):
    left = 0
    right = len(nums)

    while left < right:
        mid = left + (right - left) // 2

        if nums[mid] <= target:
            left = mid + 1
        else:
            right = mid

    return left
```

Compare carefully.

Lower bound:

```
if nums[mid] < target:
```

Upper bound:

```
if nums[mid] <= target:
```

That single equality difference changes the boundary.

---

## 19. Lower versus upper bound

For:

```
[1, 2, 2, 2, 5, 8]
```

with:

```
target = 2
```

we have:

```
lower_bound(2) = 1  
upper_bound(2) = 4
```

Therefore the target occupies:

```
[lower_bound, upper_bound)
```

or:

```
indices 1, 2, 3
```

Number of occurrences:

$$upper - lower = 3$$

---

## 20. bisect in Python

Python provides this directly:

```
from bisect import bisect_left, bisect_right  
  
nums = [1, 2, 2, 2, 5]  
  
print(bisect_left(nums, 2))    # 1  
print(bisect_right(nums, 2))   # 4
```

Conceptually:

```
bisect_left → lower bound  
bisect_right → upper bound
```

In interviews, you should still understand how to implement them yourself.

## 21. Binary search on a monotonic condition

This is the more important advanced pattern.

Suppose we are searching possible answers:

```
1 2 3 4 5 6 7 8 9 10
```

and a condition behaves like:

```
F F F F F T T T T T  
      ^
```

Once it becomes true, it remains true.

We want:

```
first True
```

Then binary search applies.

## 22. Generic first-True template

```
def first_true(left, right, condition):  
    answer = None  
  
    while left <= right:  
        mid = left + (right - left) // 2  
  
        if condition(mid):  
            answer = mid  
            right = mid - 1  
        else:  
            left = mid + 1  
  
    return answer
```

The actual problem-specific work lives inside:

```
condition(mid)
```

This pattern appears everywhere.

## 23. Example: minimum machine capacity

Imagine capacity values:

```
1 2 3 4 5 6 7 8
```

Suppose:

```
capacity 1 → insufficient
capacity 2 → insufficient
capacity 3 → insufficient
capacity 4 → sufficient
capacity 5 → sufficient
...
```

So:

```
F F F T T T T T
      ^
```

Question:

| Find minimum sufficient capacity.

This is simply:

```
find first True
```

---

## 24. Binary-search-on-answer recognition question

Ask:

| If value x works, will every larger value also work?

If yes:

```
False → True
```

monotonicity may exist.

Or:

| If x is feasible, will every smaller value also be feasible?

Then you may have:

```
True → False
```

and search for the last True.

---

## Medium Problem — Find First and Last Position of Element in Sorted Array

Given a sorted array `nums` and a `target`, return the first and last positions of the target.

If target doesn't exist:

```
[-1, -1]
```

Example:

```
nums = [5, 7, 7, 8, 8, 10]
target = 8
```

Output:

```
[3, 4]
```

---

## 25. Recognition signals

The problem gives us:

### Sorted input

```
[5, 7, 7, 8, 8, 10]
```

### Boundary requirement

Not:

```
find any 8
```

but:

```
find first 8
find last 8
```

### Expected efficient complexity

A normal linear scan gives:

[  $O(n)$  ]

but because the array is sorted and boundaries are requested:

| Binary search should immediately come to mind.

---

## 26. Brute-force reasoning

The simplest approach is:

walk left → right

When target first appears:

record first

Continue until the target disappears:

record last

Implementation:

```
def search_range_brute(nums, target):  
    first = -1  
    last = -1  
  
    for i, value in enumerate(nums):  
        if value == target:  
            if first == -1:  
                first = i  
  
            last = i  
  
    return [first, last]
```

## Correctness

Every array element is inspected.

Therefore every occurrence is discovered.

## Complexity

Time:

[  $O(n)$  ]

Space:

[  $O(1)$  ]

Correct, but it ignores the sorted property.

---

## 27. Optimized reasoning

We need two questions answered:

Where does target begin?  
Where does target end?

Use two modified binary searches:

```
Binary search 1
  ↓
find first occurrence

Binary search 2
  ↓
find last occurrence
```

Each costs:

$$O(\log n)$$

Therefore overall:

$$O(\log n) + O(\log n) = O(\log n)$$

---

## 28. Thought process

For the first position:

```
nums[mid] < target
  ↓
target must be right

nums[mid] > target
  ↓
target must be left

nums[mid] == target
  ↓
mid is a candidate
but maybe another target exists LEFT
```

For the last position:

```
nums[mid] == target
  ↓
mid is a candidate
but maybe another target exists RIGHT
```

That is the only major difference.

---

## 29. Pseudocode

```
FUNCTION find_boundary(nums, target, find_first):

    left = 0
    right = n - 1
    answer = -1
```

```

WHILE left <= right:

    mid = midpoint

    IF nums[mid] < target:
        move left boundary right

    ELSE IF nums[mid] > target:
        move right boundary left

    ELSE:
        answer = mid

        IF searching for first:
            search left half
        ELSE:
            search right half

RETURN answer

first = find_boundary(... first=True)

IF first == -1:
    return [-1, -1]

last = find_boundary(... first=False)

RETURN [first, last]

```

## 30. Python solution

```

def search_range(nums: list[int], target: int) -> list[int]:

    def find_boundary(find_first: bool) -> int:
        left = 0
        right = len(nums) - 1
        answer = -1

        while left <= right:
            mid = left + (right - left) // 2

            if nums[mid] < target:
                left = mid + 1

            elif nums[mid] > target:
                right = mid - 1

            else:

```

```
        answer = mid

        if find_first:
            right = mid - 1
        else:
            left = mid + 1

    return answer

first = find_boundary(find_first=True)

if first == -1:
    return [-1, -1]

last = find_boundary(find_first=False)

return [first, last]
```

## 31. Trace the first boundary

Input:

```
nums    = [5, 7, 7, 8, 8, 10]
target  = 8
```

Start:

```
left = 0
right = 5

mid = 2
nums[2] = 7
```

Since:

```
7 < 8
```

move right:

```
left = 3
```

Now:

```
left = 3
right = 5

mid = 4
nums[4] = 8
```

We found target.

But we're finding **first** occurrence.

Save:

```
answer = 4
```

Then search left:

```
right = 3
```

Next:

```
mid = 3  
nums[3] = 8
```

Save:

```
answer = 3
```

Search further left.

Eventually:

```
first = 3
```

---

## 32. Trace the last boundary

Again:

```
nums = [5, 7, 7, 8, 8, 10]
```

When we find 8 at index 4:

```
answer = 4
```

But this time:

```
search RIGHT
```

No later 8 exists.

Therefore:

```
last = 4
```

Final:

```
[3, 4]
```

---

### 33. Correctness condition

For `find_first`, whenever target is found at `mid`:

```
answer = mid
```

means:

| We have a valid candidate.

Then:

```
right = mid - 1
```

means:

| Search only for an even earlier valid candidate.

If none exists, the saved candidate remains correct.

The last-position search is symmetric.

---

### 34. Edge cases

#### Empty array

```
nums = []  
target = 8
```

Output:

```
[-1, -1]
```

The loop never executes.

---

#### Target absent

```
nums = [1, 3, 5, 7]  
target = 4
```

Output:

```
[-1, -1]
```

---

### One element, target present

```
nums = [8]  
target = 8
```

Output:

```
[0, 0]
```

---

### One element, absent

```
nums = [5]  
target = 8
```

Output:

```
[-1, -1]
```

---

### Every element is target

```
nums = [8, 8, 8, 8]  
target = 8
```

Output:

```
[0, 3]
```

---

This is a particularly important boundary test.

---

### Target appears once

```
nums = [1, 3, 5, 8, 10]  
target = 8
```

Output:

```
[3, 3]
```

---

### Target is at beginning

```
nums = [8, 8, 10, 15]
```

Output:

```
[0, 1]
```

---

## Target is at end

```
nums = [1, 4, 8, 8]
```

Output:

```
[2, 3]
```

---

## 35. Complexity

We perform two binary searches.

Each:

$O(\log n)$

Therefore:

**Time**

$O(\log n)$

**Auxiliary space**

$O(1)$

because the implementation is iterative.

---

## 36. Alternative solution using lower/upper bound

Once you really understand bounds, the same problem becomes elegant.

```
def search_range(nums: list[int], target: int) -> list[int]:

    def lower_bound(value: int) -> int:
        left = 0
        right = len(nums)

        while left < right:
            mid = left + (right - left) // 2

            if nums[mid] < value:
                left = mid + 1
            else:
                right = mid

        return left
```

```
first = lower_bound(target)

if first == len(nums) or nums[first] != target:
    return [-1, -1]

last = lower_bound(target + 1) - 1

return [first, last]
```

For integer targets:

```
lower_bound(target)
```

gives the first target.

And:

```
lower_bound(target + 1) - 1
```

gives the final target.

For a generic solution, an explicit `upper_bound(target) - 1` is preferable because `target + 1` only makes sense for certain target types.

---

## 37. Common binary-search mistakes

### Mistake 1 — Returning immediately on equality during boundary search

Wrong:

```
if nums[mid] == target:
    return mid
```

That finds **an** occurrence, not necessarily the first/last.

---

### Mistake 2 — Forgetting +1 or -1

Wrong:

```
left = mid
```

with a closed interval can create an infinite loop.

Example:

```
left = 4
right = 5
mid = 4
```

Then:

```
left = mid
```

leaves:

```
left = 4
```

forever.

Normally:

```
left = mid + 1
```

---

### **Mistake 3 — Mixing interval conventions**

Starting with:

```
right = len(nums) - 1
```

but using logic written for:

```
right = len(nums)
```

is a frequent source of off-by-one errors.

Know whether your interval is:

```
[left, right]
```

or:

```
[left, right)
```

---

### **Mistake 4 — Binary searching unsorted data**

This:

```
[5, 1, 9, 3, 7]
```

does not support ordinary binary search.

Why?

If:

```
nums[mid] < target
```

you cannot conclude the target lies to the right.

---

## Mistake 5 — Using binary search on answer without proving monotonicity

You need something like:

```
F F F F T T T
```

not:

```
F T F T T F T
```

Binary search fails when the condition arbitrarily switches back and forth.

---

## 38. Binary-search templates to remember

You don't need twenty templates.

Remember these three conceptual forms.

### A. Exact match

```
while left <= right:
    mid = ...

    if nums[mid] == target:
        return mid
    elif nums[mid] < target:
        left = mid + 1
    else:
        right = mid - 1
```

---

### B. Boundary search

```
answer = -1

while left <= right:
    mid = ...

    if valid(mid):
        answer = mid
        search_for_better_boundary()
    else:
        discard_invalid_half()
```

---

### C. First monotonic True

```
while left < right:
    mid = ...
```

---

```
    if condition(mid):
        right = mid
    else:
        left = mid + 1

return left
```

The third template becomes particularly important in harder interview problems.

## 39. Interview recognition hierarchy

When you see a problem, mentally ask:

1. Is the input/search space ordered?  
↓
2. Can I discard half after one comparison?  
↓
3. Am I finding:  
    exact value?  
    first/last boundary?  
    insertion point?  
    ↓
4. Or is the answer itself monotonic?  
    ↓
5. What invariant does left/right represent?

Do not begin coding until you can answer question 5.

## 40. Day 9 takeaway

Binary search is bigger than:

```
find 8 in a sorted list
```

The progression is:

```
Exact match
↓
Boundary search
↓
Lower / upper bound
↓
First True / Last True
↓
Binary search on answer
```

For today's medium problem, the central insight is:

**Finding a valid occurrence is not enough when the problem asks for a boundary.  
Save the candidate and continue searching toward the requested boundary.**

And the complexity improvement is:

```
Linear scan  
O(n)  
  
    ↓ exploit ordering  
  
Binary search  
O(log n)
```

For senior interview preparation, the highest-value skill is not memorizing `mid = ...`; it is recognizing **the monotonic property that proves half of the remaining search space can be discarded safely**.